

RAILOPT AI

Real-Data Architecture Audit and SQLite Migration Plan

Prepared from a read-only inspection of the repository. No application files were modified.

Executive summary: The project contains two incompatible SQLAlchemy model systems, two backend/API contracts, two frontend API clients, and mostly synthetic operational data. SQLite is configured as the default database, but no SQLite file was visible in the workspace during the audit. The root backend has the stronger relational design; the app backend appears to be the normal runtime path and is more synthetic/static.

1. Current Synthetic Models and Tables

- sections and stations
- assets and maintenance_tasks
- trains and train_movements
- block_windows, block_requests, optimized_blocks
- conflicts, conflict_alerts, block_plans, block_assignments
- optimization_runs and dashboard KPI values
- AI priority outputs, failure probabilities, Copilot responses, demo users

Primary sources: backend/models.py, backend/app/db/models.py, backend/data/generate_demo_data.py, backend/seed_demo.py, and backend/scripts/seed_database.py.

2. Models That Can Mostly Remain Unchanged

- User, after authentication is secured
- AuditLog
- OptimizationRun
- BlockPlan and BlockAssignment
- AI-derived fields such as ai_priority_score, ai_risk_level, failure_prob, and optimization_score

These should be labeled INTERNAL, DERIVED, or SYNTHETIC rather than official railway data.

3. Models Requiring Redesign

- Section: add station foreign keys, route/infrastructure fields, geometry, provenance, and validity dates.
- Station: add stable external ID, official code, zone/division, location quality, station type, platforms, and provenance.
- Train: replace free-text origin/destination with station relationships.
- TrainMovement: replace string-only times with date-aware, section-level intervals.
- MaintenanceTask: enforce asset-section consistency and add work location, crew, resource, and lifecycle fields.
- Blocks: separate requested, approved, executed, and optimized assignments.
- Conflicts: add foreign keys to sections, blocks, train movements, and calculation runs.

4. Required Real Station Fields

Recommended fields:

- station_code, official_name, alternate_names, external_station_id
- zone, division, state, district
- latitude, longitude, location_accuracy_m
- station_type, platform_count, running_lines_count, loop_lines_count, yard_present, junction_flag

- electrified, gauge, route_classification, operational_status
- source_name, source_url, source_record_id, retrieved_at, valid_from, valid_to, data_origin

5. Required Real Train Timetable Fields

Train service

- train_number, train_uid, train_name, train_type, operator
- origin_station_id, destination_station_id, direction
- service_days, valid_from, valid_to, priority_class
- is_passenger, is_freight, source metadata

Timetable stop

- train_service_id, station_id, sequence_number
- arrival_time, departure_time, arrival_day_offset, departure_day_offset
- scheduled_platform, distance_from_origin_km, halt_seconds, run_date, timetable_version_id

Train section movement

- train_service_id, section_id, run_date, entry_time, exit_time, direction, scheduled_or_actual, delay_minutes

6. Required Railway Section/Corridor Fields

- section_code, official_name, division_id, zone_id
- start_station_id, end_station_id, start_km, end_km, length_km
- track_count, gauge, electrification_type, signaling_system, max_speed_kmh
- directionality, route_classification, geometry, operational_status
- source metadata and validity dates

For longer routes, add corridors and corridor_sections so one train service can traverse many infrastructure sections.

7. Relationships

Trains, stations, and sections

TrainService should relate to origin_station, destination_station, timetable_stops, and train_section_movements. TimetableStop relates to TrainService and Station. TrainSectionMovement relates to TrainService and Section.

Maintenance

MaintenanceTask should relate to Asset, Section, Department, and BlockRequest. An asset should normally determine the section, with an explicit work-area override for multi-asset work.

Blocks

Use block_requests, block_request_sections, block_tasks, approved_blocks, and block_sections. A block should contain a real start/end interval and may cover multiple sections.

8. Conflict Calculation

A train/block conflict is derived when the records share a section and: $\text{block_start} < \text{train_exit}$ AND $\text{block_end} > \text{train_entry}$. Also model headway, clearance, adjacent-section, directionality, single-line, signaling, OHE, and crew/resource constraints.

Each conflict should preserve calculation_run_id, block_id, train_movement_id, section_id, overlap_start, overlap_end, overlap_minutes, severity, calculation_method, and calculated_at.

Current limitation: the root detector mainly checks overdue tasks and high train density. The app detector returns hardcoded conflicts. The simulator does not calculate timetable intervals reliably.

9. Official/Public Data Candidates

- Indian Railways public timetable publications
- CRIS/NTES public timetable or train-running information, subject to access
- data.gov.in railway datasets
- official railway zone/division and route maps
- OpenStreetMap station and railway geometry
- public electrification and infrastructure datasets
- published railway engineering standards

No external railway source is currently integrated. Imports should retain `source_name`, `source_url`, `license`, `retrieved_at`, `source_record_id`, and `source_checksum`.

10. Data That Should Remain Synthetic or Internal

- crew availability and internal work requests
- asset health and failure probabilities unless authorized source data exists
- AI priority scores and optimization outputs
- recommended blocks, projected savings, and conflict severity
- internal approvals, resource assignments, demo credentials, Copilot narratives, and fallback KPIs

11. Provenance Labeling

Use `data_origin` values: OFFICIAL, PUBLIC, DERIVED, SYNTHETIC, INTERNAL.

- OFFICIAL: authorized railway source system
- PUBLIC: public dataset
- DERIVED: calculated from other records
- SYNTHETIC: generated demo/test data
- INTERNAL: users, approvals, crew data, solver runs, and audit records

Recommended metadata: `source_system`, `source_name`, `source_url`, `source_record_id`, `source_version`, `source_checksum`, `retrieved_at`, `calculated_at`, `calculation_run_id`, `confidence_score`, `valid_from`, `valid_to`.

12. Frontend Impact

- The frontend has two data models: `frontend/src/data/mockData.ts` and `frontend/src/lib/demoData.ts`.
- It has two API clients: `frontend/src/services/api.ts` and `frontend/src/lib/api.ts`.
- Many pages directly import mock data.
- Several endpoints return static arrays or hardcoded optimization results.
- API failures silently fall back to demo values.
- Before real imports, unify API contracts, naming, types, provenance fields, demo mode, loading states, and error states.

13. Safe External Import into SQLite

- Preserve existing seed scripts as demo fixtures.
- Add staging tables: `import_batches`, `import_records`, `import_errors`.
- Store raw records before normalization.
- Map using `source_name` plus `source_record_id`.
- Validate fields, relationships, time ranges, and station sequences.
- Promote only valid records into canonical tables.
- Use source checksums for idempotency.

- Use transactions per import batch and retain import errors.
- Enable PRAGMA foreign_keys=ON, use UTC timestamps, and avoid replacing the SQLite file.
- Never use drop_all() for imports.

14. PostgreSQL/Supabase Extensibility

- Continue using SQLAlchemy and timezone-aware timestamps.
- Avoid SQLite-specific SQL and loose typing assumptions.
- Use explicit foreign keys, indexes, and stable external IDs.
- Use Alembic migrations instead of only create_all().
- Separate raw staging from canonical tables.
- Keep PostgreSQL-specific features behind adapters.
- Do not migrate to PostgreSQL during this phase.

15. Important Current Risks

- backend/run.py starts app.main:app, while the root README documents main:app.
- The two model sets use conflicting names and field types.
- The root seed expects data the generator does not create, including stations, trains, and block windows.
- The app seed does not populate many operational tables.
- The frontend expects different endpoint paths and naming conventions from the root backend.
- Dashboard and optimization metrics can appear operational even when they are synthetic.

16. Proposed SQLite-Only Migration Plan

Phase 0 - Freeze the demo

Preserve both seed scripts. Document the active runtime. Tag current rows as SYNTHETIC. Do not import external data yet.

Phase 1 - Choose the canonical model direction

Use the root relational design as the conceptual target, while isolating the app model set behind compatibility adapters until contracts are unified.

Phase 2 - Add provenance

Add data_origin, source_name, source_record_id, retrieved_at, valid_from, and valid_to to real-data entities.

Phase 3 - Normalize infrastructure

Establish canonical relationships for stations, sections, assets, and maintenance tasks. Add foreign keys and consistency validation.

Phase 4 - Normalize timetables

Split train services, timetable stops, and train section movements. Use date-aware and timezone-aware intervals.

Phase 5 - Normalize blocks

Separate requested, approved, executed, and optimized blocks. Support blocks covering multiple sections.

Phase 6 - Add import staging

Import stations first, then sections, train services, timetable stops, train movements, assets, and maintenance tasks. Make every import idempotent and transactional.

Phase 7 - Implement derived conflicts

Calculate and persist train/block, block/block, crew/resource, duration-window, and critical-task timing conflicts.

Phase 8 - Replace static APIs

Connect stations, sections, assets, tasks, movements, blocks, conflicts, and optimization outputs to the canonical database in that order.

Phase 9 - Unify frontend contracts

Retain one API client, standardize naming, remove direct mock imports, add provenance types, and keep synthetic fallback behind explicit demo mode.

Phase 10 - Harden operations

Use migrations, enable SQLite foreign keys, add temporal conflict tests, test midnight crossings, validate imports, and keep SQL portable for future PostgreSQL.

Bottom line: Keep SQLite for now, preserve the demo seeds, and introduce a canonical relational schema plus provenance and staging before importing real railway data. The first practical milestone is infrastructure and timetable normalization, not PostgreSQL migration.